

Marasanige Samarth Mahendra

+1 (857) 707-1671 · samarth.mahendragowda@gmail.com · Boston, MA

samarthmahendra.com · [github](#) · [linkedin](#) · [blog](#)

EDUCATION

Northeastern University — Khoury College of Computer Sciences

Jan 2024 – Dec 2025

Master of Science, Computer Science

Boston, MA

Courses: Programming Design · Software Engineering · Databases · Computer Systems · Algorithms · NLP+ML

TECHNICAL SKILLS

Languages: C++, C, Python, Java, SQL, NoSQL, TypeScript

Systems: Linux/Unix, Concurrency & Multithreading, gRPC, TCP/IP, POSIX sockets, epoll, B-Trees, WAL, Page I/O

Networking: VLANs, network namespaces, iptables, tcpdump, Wireshark, physical switching, routing protocols

Databases: PostgreSQL, Elasticsearch, MongoDB, Redis · query optimization, indexing, partitioning, tsvector/tsquery

Infra & Cloud: AWS, GCP, Docker, Kubernetes, Kafka, Celery, Prometheus, Grafana, Jaeger, Loki, CI/CD

Frameworks: Django REST Framework, FastAPI, Node.js · CMake, perf, Valgrind, address sanitizer

PROFESSIONAL EXPERIENCE

Northeastern University — SPHERE / NSF ■

May 2026 – Present

Research Engineer — IoT Testbed Infrastructure

Boston, MA

- Building **IoT testbed infrastructure** for **SPHERE**, an NSF mid-scale research infrastructure (USC ISI + Northeastern); ramping up on existing **Python/gRPC codebase**, **Linux network configuration**, and physical lab setup toward the goal of creating the **largest remotely-accessible IoT testbed** in the world.

Draup — B2B AI/SaaS ■

Aug 2022 – Nov 2023

Associate Software Development Engineer

Bengaluru, India

- Led **PostgreSQL → Elasticsearch** migration for analytics and real-time aggregation APIs over **800M+ documents**; applied **custom indexing, range partitioning, and materialized views** delivering **3× faster analytics API response times** and **50% lower operational costs** at production scale.
- Owned **5 PostgreSQL → Elasticsearch API migrations** as part of a team-wide split; refactored query functions where necessary while preserving **zero breaking changes** in all API contracts.
- Delivered **Account Insights comparisons module** — aggregation APIs with **GROUP BY, window functions, and per-filter percentile buckets**; handed over with full **API documentation** and data model walkthroughs.
- Led **University hiring module revamp**: **12 dynamic filters** with **custom tsvector/tsquery full-text engine** + Boolean parser; reduced data volume from **5yr → 1yr** with gateway team + **Redis TTL caching** — **~5× p90 latency improvement** on filtered queries.
- Owned **Outsourcing filters revamp**: refactored tightly coupled legacy code into **clean modular architecture**; refactored **3 filters**, added **2 new ones**; built **ecommerce dead-end prevention** — cut filter dev time from **days to hours**, caught silent UX blocker pre-production.
- Built two **Django admin tools**: **feature-flag system** (Sales/CS toggle product tiers without deploy) + **Celery ES-push job dashboard** (stats, retry counts, logs) — **~2× faster job failure detection and resolution** vs. manual log-grep.
- Took **sole ownership** of **Digital Tech Stacks** analytics after owner left — audited and corrected **silent aggregation logic errors** across all APIs; established **ownership documentation** preventing future regressions.
- Self-initiated **ChatGPT plugin POC** → **shipped as product feature**; built **AutoGPT-style eng copilot** on CTO request; delivered **CPO-requested talent metrics pipeline** (Django mgmt cmd + Jenkins, **100K+ role/company combinations**, batched offset/limit saves for fault tolerance).

Draup ■

Apr 2022 – Jul 2022

Software Development Engineer Intern

Bengaluru, India

- Implemented **Redis binary image caching**, eliminating redundant S3 fetches; presented at **internal eng brown bag**; built **Datadog + CloudWatch** dashboards; automated **Jenkins CI/CD** + DB cleanup jobs.

PROJECTS · INDEPENDENT WORK

JobHarvester — Distributed AI-Driven Job Ingestion Platform ■

Jan 2025 – Mar 2026

[Python](#) · [AsyncIO](#) · [FastAPI](#) · [Kafka](#) · [PostgreSQL](#) · [Redis](#) · [Kubernetes](#) · [Prometheus](#) · [Jaeger](#) · [OpenTelemetry](#)

- Architected **cloud-native ingestion platform** on **Kubernetes** using **Kafka Partitioning + Consumer Groups**; containerized **Producer/Consumer/Monitoring pods** via **Docker + Minikube** — designed to scale across **10,000+ domains** at **~50–120 pages/min/node** with **sub-second queue latency**.
- Engineered **GPT-4o + Pyppeteer** autonomous CSS selector extraction (**95% less manual config**) with **self-correcting DOM validation loop + anti-detection layer** (UA rotation, header spoofing, viewport randomization) — **~20–30% higher** scrape success on bot-protected SPAs.
- Reliability**: DLQ for failed scrapes + **Token-Bucket Rate Limiters** (prevents IP blacklisting) + **PostgreSQL pooling** (50+ pods) + **URL-based dedup** (**100% accuracy**) — **99.9% uptime**. **Observability**: **Prometheus · Jaeger · Loki · OpenTelemetry** for metrics, tracing, and logs across the cluster.

Orion — Dependency-Aware Parallel Task Execution Framework ■

Dec 2025 – Feb 2026 (personal project)

[C++23](#) · [gRPC](#) · [Protocol Buffers](#) · [DAG scheduling](#) · [CMake](#) · [distributed systems](#)

- Designed a **Ray-style distributed task runtime** in **C++23**, modeling computations as **DAGs** to enable **horizontally scalable execution** from a single-process engine to a **multi-node cluster**; built and packaged via **CMake**.
- Engineered a **distributed cluster scheduler** using **gRPC + Protocol Buffers** for RPCs, with **node discovery** and **cross-node dependency tracking** to minimize scheduling latency and dispatch tasks upon **data readiness**.
- Implemented **lock-based concurrency** with **condition variables** and **work queues**, ensuring **deterministic execution** and **zero busy-waiting** under load; validated with **address sanitizer** and stress tests.